

```

decrpt.py  Encrypt.py
1  from cryptography.fernet import Fernet
2
3  # Generate a key for encryption and decryption
4  key = Fernet.generate_key()
5  cipher_suite = Fernet(key)
6
7  # Function to encrypt and save text data to a locked file
8  def encrypt_and_save(text_data, filename):
9      encrypted_data = cipher_suite.encrypt(text_data.encode())
10
11      with open(filename, 'wb') as file:
12          file.write(encrypted_data)
13
14      print(f"Text data encrypted and saved to {filename}")
15
16 # Example usage: Encrypt and save text data to a locked file
17 text_to_encrypt = "This is a secret message."
18 encrypted_file = 'locked_file.txt'
19
20 encrypt_and_save(text_to_encrypt, encrypted_file)
21

```

Fig. 3: Code snippet for encrypting the data

```

upt.py  Encrypt.py
1  from cryptography.fernet import Fernet
2  import rdm6300
3
4  # Generate a key for encryption and decryption
5  key = Fernet.generate_key()
6  cipher_suite = Fernet(key)
7
8  # RFID card chip id to allow decryption
9  allowed_chip_id = 53346
10
11 # Function to unlock and decrypt the data from a locked file
12 def unlock_and_decrypt(filename, rfid_chip_id):
13     with open(filename, 'rb') as file:
14         encrypted_data = file.read()
15
16     if rfid_chip_id == allowed_chip_id:
17         decrypted_data = cipher_suite.decrypt(encrypted_data).decode()
18         print(f"Decrypted Data: {decrypted_data}")
19     else:
20         print("Unauthorized access. The correct RFID card is required for decryption.")
21
22 # Function to scan RFID card chip
23 def scan_rfid():
24     reader = rdm6300.Reader('/dev/ttyS0')
25     print("Please scan your RFID card...")
26     while True:
27         card = reader.read()
28         if card:
29             return card.value
30

```

Fig. 4: Code snippet for decrypting the data

data on the chip, and a compatible scanner attached to the USB of the Raspberry Pi or laptop is used to read the data inside the implanted chip. In the second method, the chip contains the password as a UUID, and data stored in the cloud or laptop can be accessed using the chip scan.

Fig. 1 shows some implantable chips and Fig. 2 captures the pro-

cess of implanting the chip inside an orange segment for testing purposes. The implantable chip on the right side of the image in Fig. 1 possesses features that include memory for data storage and an LED that illuminates upon scanning. It boasts a comprehensive set of features, outlined as follows:

(a) 13.56MHz ISO14443A & NFC Type 2 chip

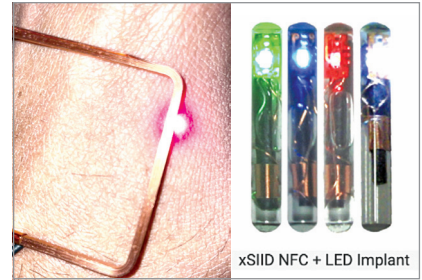


Fig. 5: Implanted chip LED glowing when scanned inside the body

```

Shell
>>> %Run Encrypt.py
Text data encrypted and saved to locked_file.txt
>>>

```

Fig. 6: Python shell output on encryption of the message and data

- (b) 2x15mm cylindrical sterile bioglass implant
- (c) 1kB (888 bytes) of user writable memory
- (d) Integrated power harvesting LED indicator

These features collectively contribute to the chip's effectiveness and versatility, making it suitable for secure data storage within the human body.

Software coding

Initially, we create a code that locks and encrypts the actual data, ensuring it remains unreadable even if someone accesses the encrypted data. The data becomes readable only when scanned with the implanted chip. While we have

EFY Note

The source code of this project is available for free download at www.electronicsforu.com